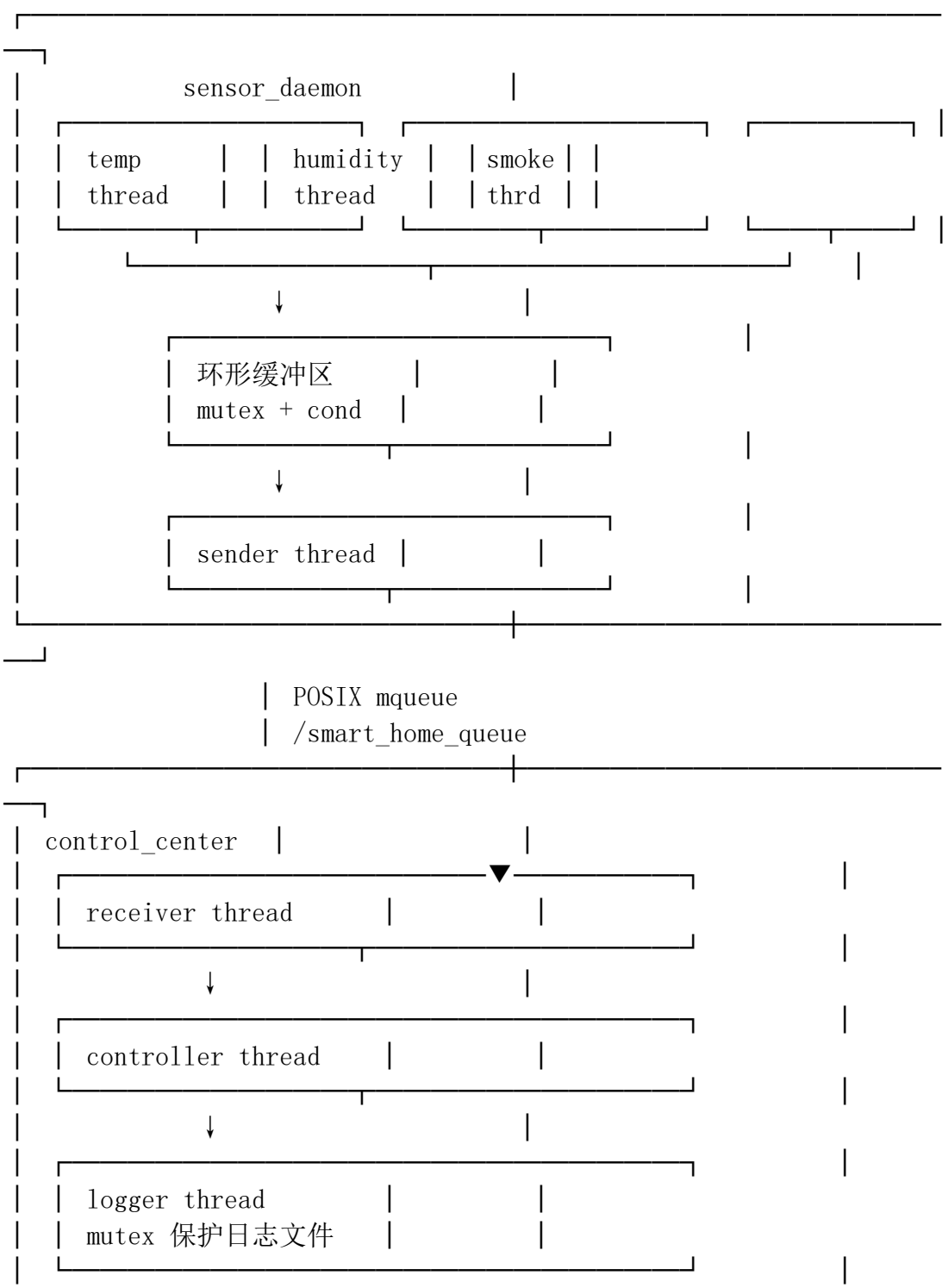
智能家居模拟系统实验报告

一、系统架构图与数据流说明

1. 系统架构图（进程 / 线程）



└──┘

2. 数据流说明

传感器线程周期性生成数据

数据写入共享环形缓冲区

发送线程从缓冲区取出数据

通过 POSIX 消息队列发送到控制中心

控制中心接收线程解析消息

控制线程根据规则执行动作

日志线程将事件写入 smart_home.log

二、关键数据结构及同步方案设计

1. 消息结构体（进程间通信）

```
typedef struct {
    char type[16];      // temperature / humidity / smoke
    float value;
    time_t timestamp;
} sensor_msg_t;
```

设计原因：

统一数据格式

便于 mq_send / mq_receive 传输

支持扩展更多传感器类型

2. 环形缓冲区（生产者 - 消费者）

```
typedef struct {
    sensor_msg_t buf[BUFFER_SIZE];
```

```
    int in;
    int out;
    pthread_mutex_t mutex;
    pthread_cond_t not_full;
    pthread_cond_t not_empty;
} ring_buffer_t;
```

成员	作用
buf	存储传感器数据
in / out	生产 / 消费指针
mutex	保护缓冲区
not_full	防止写溢出
not_empty	防止读空

3. 同步机制使用位置与原因

（1）传感器进程

场景	同步方式	原因
多线程写缓冲区	mutex	防止数据竞争
缓冲区满	cond（not_full）	避免忙等
缓冲区空	cond（not_empty）	唤醒消费者

使用 while 判断条件，防止虚假唤醒

（2）控制中心进程

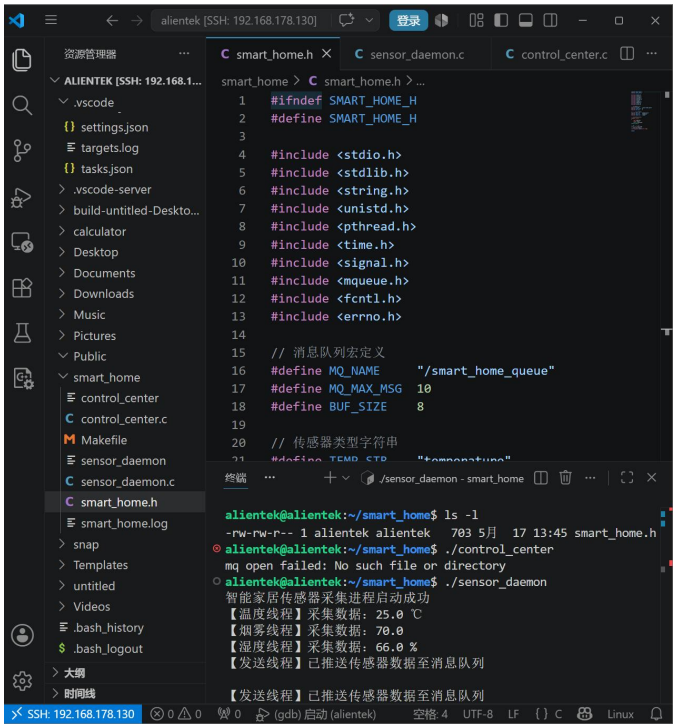
场景	同步方式	原因
写日志文件	mutex	保证日志顺序
多线程写同一文件	mutex	防止内容交错

（3）进程间通信

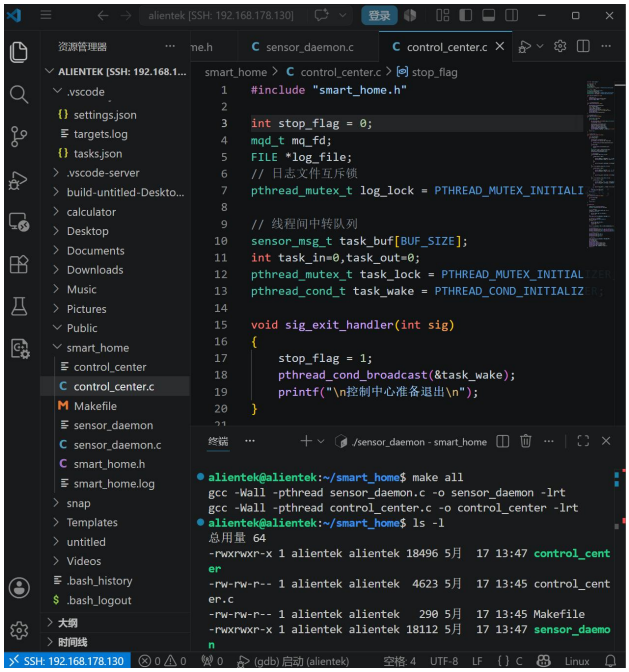
技术	原因
POSIX 消息队列	解耦进程、支持异步
mq_open / mq_send	标准 IPC 接口
统一消息结构	易解析、可扩展

三、运行结果示例

1. 控制台输出示例



2. 日志文件 smart_home.log 内容

3. 

四、遇到的问题及解决办法

在实验过程中，我遇到了几个典型的多线程与 IPC 问题。
消息队列一度无法打开，后来确认是权限问题，改用统一队列名并正确挂载 /dev/mqueue 后解决。
日志文件曾出现内容交错，我通过增加 pthread_mutex_t 保证了线程安全。
环形缓冲区曾发生死锁，将条件变量的 if 判断改为 while 后问题解决。
最初使用 Ctrl+C 退出时资源未释放，我注册了 SIGINT 信号处理函数进行清理。
此外，多线程随机数重复，改用 rand_r()并为每个线程设置不同种子后恢复正常。

五、信号量与条件变量对比分析

对比项	条件变量	信号量
是否需要 mutex	必须	可选
表达能力	强（复杂同步）	弱（计数）
可读性	高	中
易出错点	虚假唤醒	忘记 sem_post
适用场景	缓冲区同步	资源计数

本实验选择条件变量：

原因是需要同时表达 **“缓冲区空 / 满”** 两种状态，语义更清晰。

六、总结

本实验成功实现了基于多线程与进程间通信的智能家居模拟系统。

通过 pthread 实现了并发数据采集与控制，使用互斥锁与条件变量保证了线程安全，

并通过 POSIX 消息队列完成了进程间数据交换，达到了作业要求的各项技术指标。